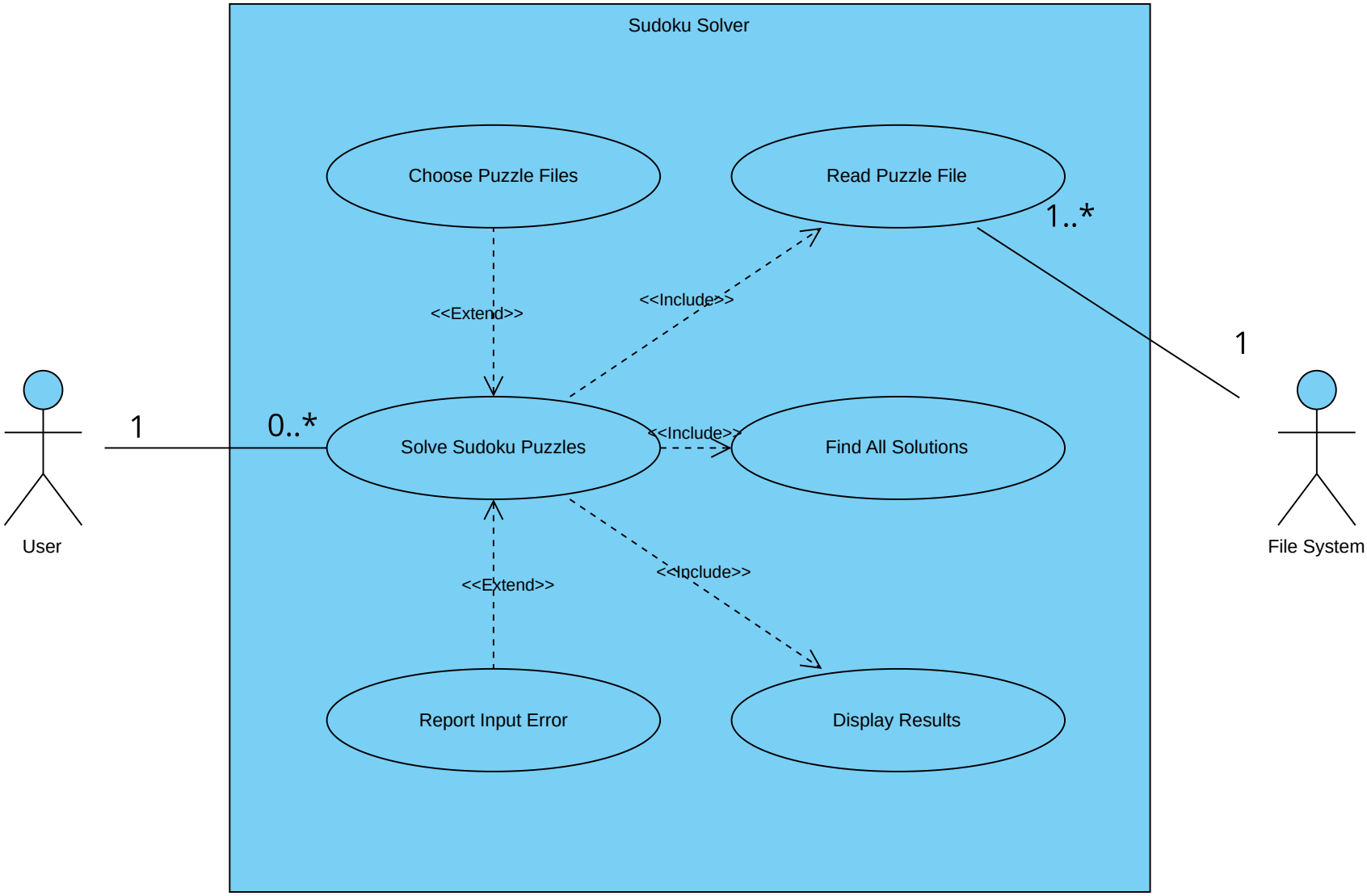
Made with VisualParadigm
For non-commercial use



Sudoku Solver - Use Case Notes

Matthew Bisbee | EECS 348 Assignment 4

1. Identify the use case

Use case: Solve Sudoku Puzzles
System: Sudoku Solver (Assignment4.exe)
Actors: User and File System

The User runs the solver to see the original puzzles and all possible solutions. The File System provides the puzzle files. No puzzle files are changed.

2. Brief description

Read puzzle1.txt thru puzzle5.txt by default, or use filenames given by the User. For each file, print its name and original board, then all valid solutions. Print "No solution found" if none exist. Report input errors separately and continue to the next file.

3. Step-by-step flow

1. User starts the solver, optionally giving filenames.
2. System chooses the given files or the five defaults.
3. System gets the next file from the File System.
4. System checks its format and displays the original board.
5. System checks clues and finds all solutions.
6. System displays each solution and the total, or no solution.
7. System repeats for the remaining files.
8. System finishes and returns its exit status.

Relationships shown in the diagram

Read Puzzle File, Find All Solutions and Display Results are included in the normal flow. Choose Puzzle Files applies when filenames are given. Report Input Error applies when a file cannot be read or has invalid entries. These alternatives are described below.

4. Detailed description

Before starting: The program is available on a Cycle server. The User has the puzzle filenames, or runs it from the folder containing the five default files.

Basic flow

1. Start and choose files

User runs ./Assignment4.exe. With filenames, the system processes those files in order. Otherwise it uses puzzle1.txt thru puzzle5.txt.

2. Read and check input

System displays the filename and requests its contents from the File System. Require nine nonblank rows with nine entries each. Digits 1-9 are clues; underscores are blanks. Blank lines, trailing spaces and Windows line endings are accepted.

3. Display the original puzzle

System prints the original nine rows with blanks shown as underscores. The given clues stay unchanged while finding solutions.

4. Find and display results

System finds every valid completion, rather than stopping at the first. Each completed row, column and 3x3 box must contain 1-9 once. Print each solution and the total count. If no completion exists, print "No solution found".

5. Continue and finish

Repeat for each remaining file. After all files are processed, return 0 if input and output succeeded, or 1 if any file had an input error. Files with input errors do not prevent later files from being processed.

Alternative flows

Missing or malformed file: Print an error explaining the problem, skip that file and continue. Do not report this as an unsolvable puzzle.

Conflicting clues or no solution: Report no solution and continue to the next file. A well-formatted puzzle with no solution is not an input-format error.

Output failure or unexpected error: Report the failure on the error stream and stop. Return 2 for failed output or 3 for an exception. Some files or results may be incomplete.

After normal completion: Every requested file has a result or input error. Each valid puzzle shows its original board and all solutions, or no solution. Input files stay unchanged. These complete-result guarantees do not apply if the run stops early.